

Singly Linked List (Basic Operations)

(Insertion at the end)

```
void insertatend()
```

```
{
```

```
    cout << " ";
```

```
    int val;
```

```
    // value to insert
```

```
    cin >> val;
```

```
    Node* newnode = new Node(val);
```

```
    // creating newnode
```

```
    if (head == NULL)
```

```
    {
```

```
        head = tail = newnode;
```

```
    }
```

```
    // if there is no node before, assign  
    both head & tail → newnode
```

```
    else
```

```
    {
```

```
        tail->next = newnode;
```

```
        tail = newnode;
```

```
    }
```

```
    // if not 1st node, then put it in the next  
    tail & assign tail this newnode.
```

```
}
```

(Insertion at the beginning)

void insertionatthebeginning()

```
{
    // Rest of the simple code
    if (head == NULL)
    {
        head = tail = newnode;
    }
    // if first node
    else
    {
        newnode->next = head;
        head = newnode;
    }
    // if not 1st, then move head to the next
    // of this newnode & then point head
    // to this newnode
}
```

(Insertion after value)

void insertionaftervalue()

```
{
    if (head == NULL)
    {

```

```
        return;
    }

```

// if list is empty

Node* temp = head; // To traverse to find value without changing head.


```
while(temp != NULL && temp->data != target)
```

```
{
```

```
    temp = temp->next;
```

```
}
```

// moving temp to the target & exiting loop if got NULL or target.

```
if(temp == NULL)
```

```
{
```

```
    return;
```

```
}
```

// to check if loop exits bcz we got NULL so then return (No val found).

```
Node* newnode = new Node(val);
```

// created newnode with new val

```
newnode->next = temp->next;
```

// moving next of temp to the next of newnode as it will be inserted between them to and we have to link them.

```
temp->next = newnode;
```

// now moving newnode in the next of temp.

```
if(temp == tail)
```

```
{
```

```
    tail = newnode;
```

```
}
```

// if temp reaches end, point tail to newnode.

```
}
```

(Delete by Value)

```
if(head == NULL)
```

```
return;
```

```
}  
// if list is empty
```

```
if(head->data == val)
```

```
// if value is at head
```

```
Node* temp = head;
```

```
// move head to temp
```

```
head = head->next;
```

```
// point head to its next
```

```
if(head == NULL)
```

```
// if head->next was NULL  
(currently head)
```

```
{
```

```
tail = NULL;
```

```
// move tail tail, NULL
```

```
}
```

```
delete temp;
```

```
// delete that node
```

```
return;
```

```
}
```

```
Node* prev = NULL;
```

```
// to track the data of prev node  
of the (to be deleted node)
```

```
Node* temp = head;
```

```
// to traverse & find value  
without changing head
```

```
// prev will always be at 1 node before temp
```

```
while(temp != NULL && temp->data != val)
```

```
{
```

```
prev = temp;
```

```
// traversal to find val to  
delete & point temp  
to that node.
```

```
temp = temp->next;
```

```
}
```



```

if (temp == NULL) // if we get temp == NULL
                    return it.
{
    return;
}

```

```

prev->next = temp->next;

```

as prev points to 1 node before temp that is to be deleted so in order to link the prev & next of temp we do this.

```

if (temp == tail) // if temp was tail
{
    tail = prev; // point tail to prev
}

```

```

delete temp; // delete temp
}

```

(Search by Value)

```

bool searchbyvalue()
{

```

```

    if (head == NULL) // if no node
                        list empty
    {
        return false;
    }

```

```

Node* temp = head; // same scene

```

```

while (temp != NULL) // For traversing
{

```

```

if (temp->data == val)           // if found data
{
    cout << "-";                // prompt data
    return true;                // return true
}
temp = temp->next;               // else, increment temp
}
return false;                   // if not found return false
}

```

(Void Update)

```

void update()
{
    ....
    Node* temp = head;          // Prompt & input
                                // same scene
    while (temp != NULL)        // traverse
    {
        if (temp->data == val)   // if val to be updated
        {                       found.
            temp->data = newval; // update val
            return;
        }
        temp = temp->next;       // increment
    }
}

```

(Void Display)

1) Simple travers &
 1) if no node, return

(DRY_RUN)

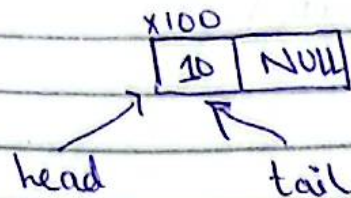
Insertion at end:

⇒ First node entry: (10 enter ⇒ data)

head = NULL

tail = NULL

after newnode:



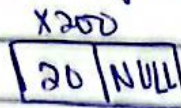
// this is when both head = tail = NULL

⇒ When 2nd or every new entry:

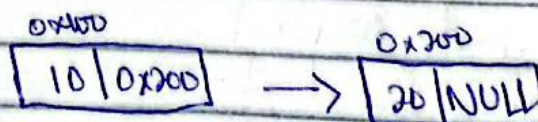
head = 0x100

tail = 0x100

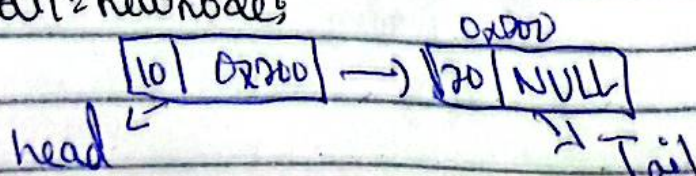
New newnode entry: (20 enter)



tail → next = newnode:



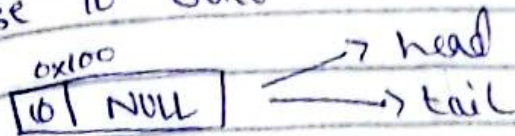
tail = newnode:



insertatbeginning():

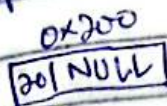
// if (head == NULL) { -- }

When no node before
suppose 10 enters

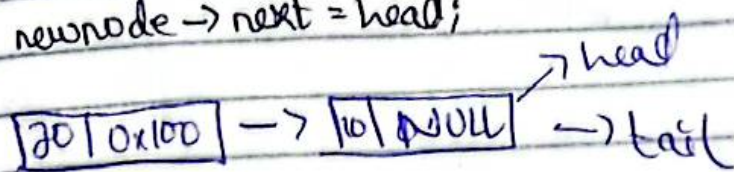


// else { -- } -- }

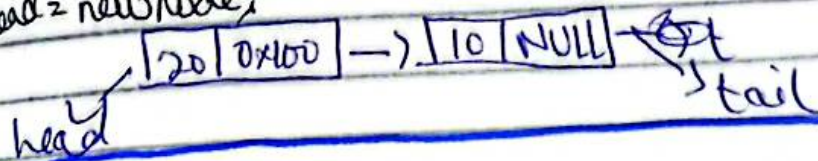
When 2nd or more entries
suppose 20 enters



Now: newnode → next = head;



head = newnode;



insertatvalue():

• When head = NULL, means no node before
so return

• Node* temp = head; // so we can traverse through
the list & ~~find~~ starting from head
without changing head to find value.

while(1) {

Suppose this is our list

head	10	0x200
0x200	20	0x300
0x300	30	0x400
0x400	40	0x500
0x500	50	NULL

temp → 0x200 // We will traverse it
we find value (30)

// here it is at data = 10
temp ≠ NULL. s.
// so,

temp = temp → next
that is 20

// again temp ≠
at temp = 0x300

temp ≠ NULL ✓
data ≠ target ✓

loop
continue

So,

temp = temp → next;

at temp = 0x300

temp ≠ NULL ✓
data = target X

loop exit

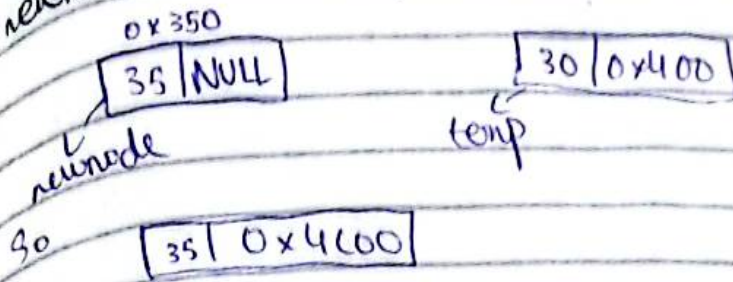
~~if (temp == 1)~~

if (1) // as temp isn't NULL

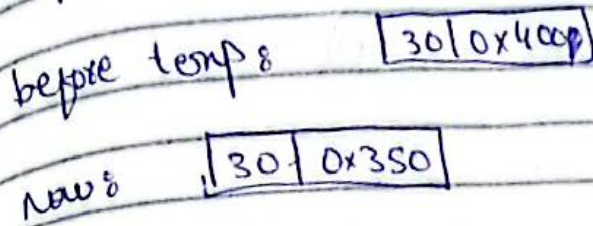
Note newnode = new Node(Val); say

35 | NULL

1. newnode $\rightarrow$ next = temp $\rightarrow$ next;



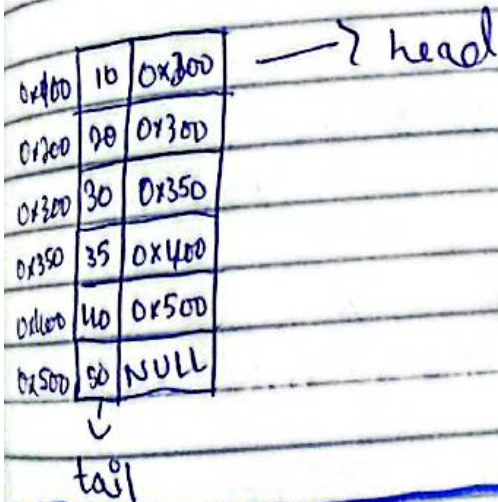
2. temp $\rightarrow$ next = newnode;



3. if (temp == tail): not tail

// if tail then also update tail

4. New linked list is:



Delete by value():

if (head == NULL): // if no node return

Suppose:

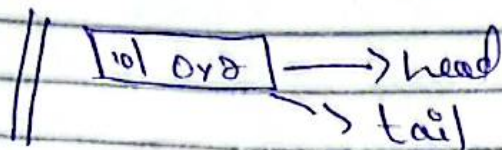
Our list is:

0x1	10	0x2	→ head
0x2	20	0x3	
0x3	30	0x4	
0x4	40	0x5	
0x5	50	NULL	→ tail

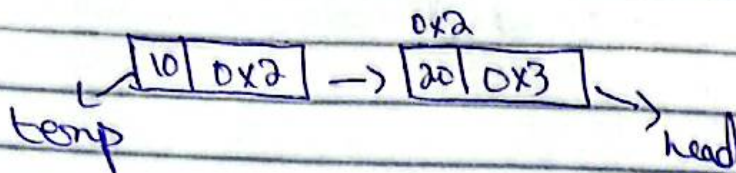
• Lets delete 0x1 → (10)

if (head → data == val) ✓
{

Node* temp = head; // new node created



head = head → next



if (head == NULL) X
{ }

~~delete~~ delete temp;

return;

}

New list is:

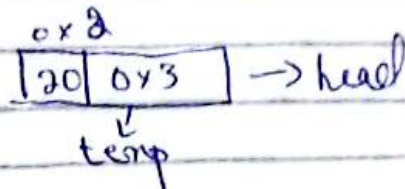
0x2	20	0x3	→ head
0x3	30	0x4	
0x4	40	0x5	
0x5	50	NULL	→ tail

1) Lets delete (30);

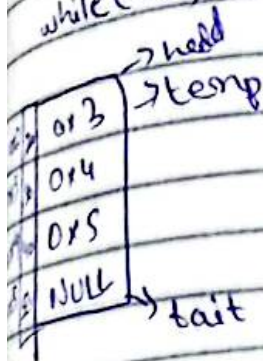
if (head->data == Val) X

Node* prev = NULL;

Node* temp = head;



while (---)



if (temp == 0x2):

temp = NULL ✓
data = Val ✓
temp = temp->next;

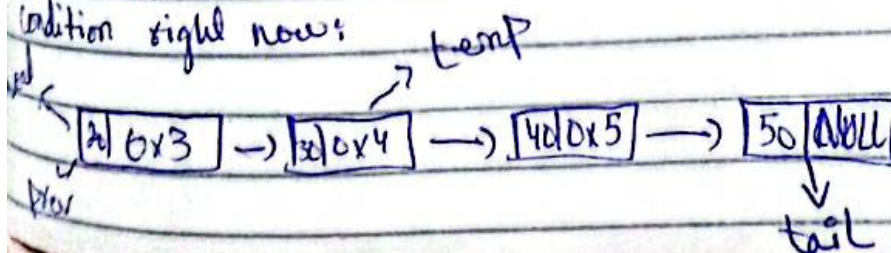
prev = temp

if (temp == 0x3)

temp = NULL ✓
data = Val X

exit loop;

condition right now:

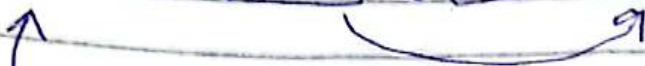


if (temp == NULL) {
3

1) prev -> next = temp -> next;

before: [20|0x3] -> [30|0x4] -> [40|0x5] -> [50|NULL]

Now: [20|0x4] -> [30|0x4] -> [40|0x5] -> [50|N]



if (temp == tail) {
3}

delete temp;

So,

before:

Now: [20|0x4] -> [40|0x5] -> [50|NULL]

void Update();

1) Node* temp = head;

Suppose; Our linked list is:

0x1	10	0x2	→ head, temp
0x2	20	0x3	
0x3	30	0x4	
0x4	40	0x5	
0x5	50	NULL	→ tail

while(—);

if temp = 0x1;

if (temp->data == val) X
temp = temp->next;

updating
finding (30) to (25)

if temp = 0x2;

if (temp->data == val) X
temp = temp->next;

if temp = 0x3;

if (temp->data == val) ✓
{
temp->data = newval;
return;
}

So, updated list is:

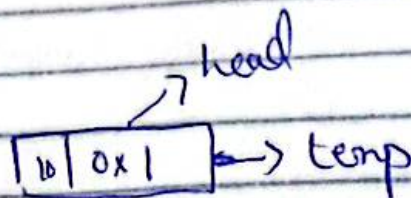
0x1	10	0x2
0x2	20	0x3
0x3	25	0x4
0x4	40	0x5
0x5	50	NULL

bool Searchbyvalue():

lets search 25 in :

if (head == NULL) { } X

Node* temp = head;



while(→) at temp → 0x1

• ~~temp = NULL~~ ✓

⑥ if(temp → data = val) X
temp = temp → next;

at t = 0x2;

if(t → data = val) X
t = t → ~~data~~ next;

at t = 0x3;

if(t → data = val) ✓
{

cout << "→";

return true;

}

}

~~X~~ → ~~X~~

•) Void display():

Suppose:

0x1	1	0x2
0x2	2	0x3
0x3	3	0x4
0x4	4	NULL

→ head / temp
~~Node*~~ temp = head
→ tail

while(temp ≠ NULL)

{

cout << temp → data;

temp = temp → next;

}

(SLL-Questions)

(Question # 01)

Void ReverseList():

```
{  
    if(head == NULL || head->next == NULL)  
    {  
        return;  
    }  
}
```

//return if no node or only 1 node as
no need to reverse it

Now: //if more than 1 node

```
Node* prev = NULL;    tail = head; //point  
Node* curr = head;    to head  
Node* next = NULL;
```

//we will use these pointers for shifting
nodes.

```
while(curr != NULL) //run loop until curr = null  
{  
    next = curr->next; //move curr's next into next  
    curr->next = prev; //move prev into curr's next  
    prev = curr; //point prev to curr  
    curr = next; //point/assign next to curr  
}
```

```
head = prev; //at the end point head to prev  
}
```


(DRY RUN)

Suppose Our Linked List is

Now
Before iterations:

tail $\rightarrow$ 0x1
prev = NULL
curr $\rightarrow$ 0x1 (head)
next = NULL



Now
(curr $\neq$ NULL) ✓

```

{
    next = curr -> next;
    curr -> next = prev;
    prev = curr;
    curr = next;
}

```

next = 0x2
0x1 $\rightarrow$ next = NULL
prev = 0x1
curr = 0x2



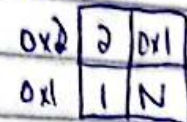
again:

```

while (curr  $\neq$  NULL) ✓
{
    next = curr -> next;
    curr -> next = prev;
    prev = curr;
    curr = next;
}

```

next = 0x3
0x2 $\rightarrow$ next = 0x1
prev = 0x2
curr = 0x3



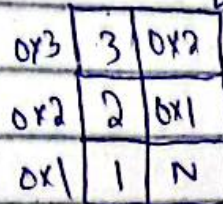
again:

```

while ( ) ✓
{
    next = curr -> next;
    curr -> next = prev;
    prev = curr;
    curr = next;
}

```

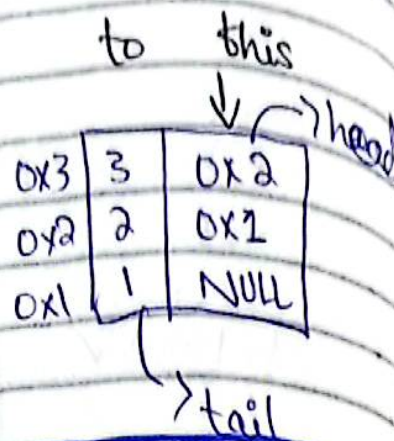
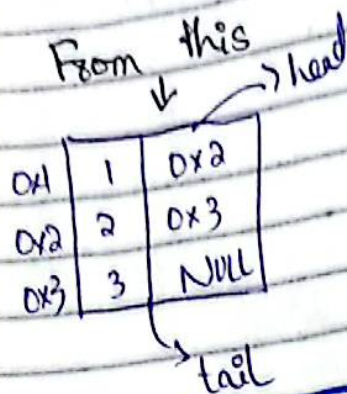
next = ~~0x4~~
0x3 $\rightarrow$ next = 0x2
prev = 0x3
curr = ~~0x4~~



~~ahead = prev;~~
 while (curr != NULL) X

l = 3; // point head to prev
 head = prev;

So our list became:



(Question # 02)

void merge(SLL &L2)

{
 Node* h1 = head; // head of L1
 Node* h2 = L2.head;

if (h1 == NULL) // if L1 is empty
 {
 head = h2;
 tail = L2.tail;
 return;
 }

if (h2 == NULL) // if L2 is empty
 {
 return;
 }


```

// for new merged list
newhead = NULL;
newtail = NULL;

// for sorted list
if (h1->data <= h2->data)
{
    newhead = newtail = h1;
    h1 = h1->next;
    // point both to h1;
    // to save head
}
else (h2->data < h1->data)
{
    newhead = newtail = h2;
    h2 = h2->next;
    // for initial head & tail
}

while (h1 != NULL && h2 != NULL) // run until any head = NULL
{
    if (h1->data <= h2->data)
    {
        newtail->next = h1;
        newtail = h1;
        h1 = h1->next;
        // link to newtail
        // update newtail
        // iterate h1
    }
    else (h2->data < h1->data)
    {
        newtail->next = h2;
        newtail = h2;
        h2 = h2->next;
        // Same
        // Same
        // iterate h2
    }
}

if (h1 != NULL)
{
    newtail->next = h1;
}
if (h2 != NULL)
{
    newtail->next = h2;
}

// in case if loop finishes
// bcz of any h1 or h2 so
// link the remainings of any
// to next of newtail

```

```
while(newtail->next != NULL)
```

// move tail to

```
{
    newtail = newtail->next;
}
```

```
head = newhead;
tail = newtail;
```

// update head & tail
new of merged

(DRY RUN)

Suppose: our 2 lists are:

0x1	1	0x3
0x3	3	0x5
0x5	5	N

0x2	2	0x4
0x4	4	N
0x6	6	N

L2

h1 = head

h2 = h2-head

h1 → 0x1

h2 → 0x2

newhead = NULL;

newtail = NULL

as h1 < h2 so

newhead = newtail = 0x1

Now:

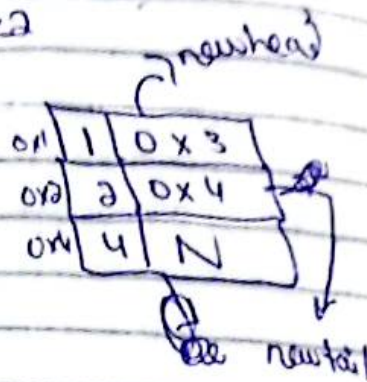
h1 = h1->next;

h1 = 0x3

while (h1 != N && h2 != N) ✓

if (h1->d < h2->d) X 3 < 2

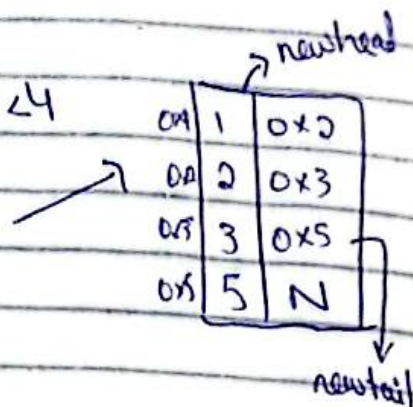
else {
newtail->next = h2;
newtail = h2;
h2 = h2->next;
}



again: while (-) ✓

if (h1->d <= h2->d) ✓ 3 < 4

{
newtail->next = h1;
newtail = h1;
h1 = h1->next;
}

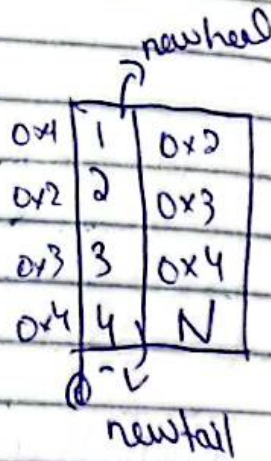


else { ~ }

again: while (-) ✓

if (-) X 5 <= 4

else {
newtail->next = h2;
newtail = h2;
h2 = h2->next; }

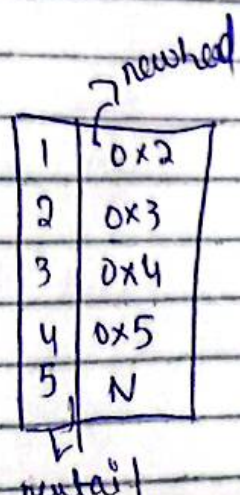


while (-) X h2 = NULL

if (h2 == NULL) ✓

{
newtail->next = h1;
}

if (h2 != NULL) X



```
head = newhead;
tail = newtail;
```

// updated head & tail
(L-2).

(Question #03)

```
void findmiddle()
```

```
{
    Node* slow = head;
    Node* fast = head;
```

// point both to head
traversal

```
while(fast != N && fast->next != N)
```

```
{
    slow = slow->next;
```

```
    fast = fast->next->next;
```

// iterate 1 step
// iterate 2 steps

```
}
```

```
}
```

(DRY - RUN)

Suppose: our list is

fast = 0x1

slow = 0x1

0x1	1	0x2
0x2	2	0x3
0x3	3	0x4
0x4	4	0x5
0x5	5	N

head

tail

```
Now: while(fast != N && fast->next != N)
```

```
{
    slow = slow->next;
```

```
    fast = fast->next->next;
```

```
}
```

slow = 0x2

fast = 0x3

```
while(—) ✓
```

```
again: { slow = slow->next;
```

```
        f = f->next->next;
```

```
}
```

s = 0x3

f = 0x5

while(→) X fast → next = NULL

data is at slow, 0x3 → 3

(Question # 04)

void deleteatend()

Node* fast, *slow = head;

for(int i=0; i<n; i++) // move fast to that
point if exist
{
 —; }
}

if(fast == NULL) // if n reaches head
{
 Node* del = head; // save head
 head = head → next;
 delete del; // delete
 return;
}

while(fast → next != NULL)
{
 fast = fast → next; // move towards the node
 slow = slow → next; // to be deleted
}

Node* del = slow → next;
slow → next = slow → next → next; // move slow to del
next
if(slow → next == NULL) // if ~~fast is deleted~~
{ tail = slow; } delete del;

(DRY RUN)

Suppose, our list is

0x1	1	0x2
0x2	2	0x3
0x3	3	0x4
0x4	4	N

fast = 0x1
slow = 0x1

let us enter 2:

1) for(int i=0; i<n; i++) ✓
{
if(fast == NULL) X
fast = fast->next; f = 0x2
}

2) i=1, i<2; ✓
if(—) X
fast = fast->next; f = 0x3

3) i=2, i<2 X

4) if(fast == NULL) X

Now while(fast->next != NULL) ✓
{

fast = fast->next; f = 0x4
slow = slow->next; s = 0x2
}

Again while(—) X f->next = NULL

Node* del = slow->next; del -> 0x3

and ——— 2nd last

RDC

(Question 05)

void createcycle(int pos)

{
if (pos < 0)
return;
}

// if negative val

Node* temp = head;

// to traverse

Node* cycnode = NULL;

// to find node for cycle

int count = 0;

while (temp != NULL)

// run loop until temp = NULL

{
if (count == pos)

// if pos found

{

cycnode = temp;

// save that node

}

if (temp->next == NULL)

// if last node

{

break;

}

temp = temp->next;

// increment node

count++;

// increment count

}

if (cycnode != NULL && temp != NULL)

{

temp->next = cycnode;

// point next of tail
to that node

}

}

bool hasCycle()

```
{
    Node* slow = head;
    Node* fast = head;
```

// for traversing

```
while (fast != NULL && fast->next != NULL)
{
```

```
    slow = slow->next;
```

```
    fast = fast->next->next;
```

// i+1;
// i+2;

```
    if (slow == fast)
```

```
    {
        return true;
```

// mean if tail == head
then fast will be at head
a loop & will never be null

if yes then return

```
    }
    return false;
```

→ otherwise false.

(DRY_RUN)

0	1	2	3
0x1	0x2	0x3	0x4
1	2	3	4

Suppose: List is

Suppose pos = 2

createCycle();

temp = 1 (0x1)

```
1) while (temp != NULL) ✓
{
```

```
    if (count == pos) X 0 ≠ 2
```

```
    if (t->next == NULL) X
```

```
    temp = t->next;
```

t = 0x2

```
    c++;
```

c = 1

```
}
```


while(→) ✓

```
if (c == pos) 1 ≠ 2 X
if (t → n = N) X
t = t → next;
c++;
```

t = 0x3
c = 2

while(→) ✓

```
if (c == pos) 2 = 2 ✓
```

cycnod = temp;

cycnod = 0x3

```
if (t → next = NU) X
temp = t → next;
c++;
```

t = 0x4

while(→) ✓

```
if (→) X
if (t → n = N) ✓
{ break; }
```

cycle created

```
if (cycnod != N && t != N) ✓
```

temp → next = cycnod;

0x1	1	0x2
0x2	2	0x3
0x3	3	0x4
0x4	4	0x3

cycle created.

Question # 06

```
Node* gethead() // to access head
{
    return head;
}
```

```
SSL addition(SSL & L1, SSL & L2) // 2 arguments of :
{
    SSL result; // addition result
```

```
Node* h1 = L1->gethead(); // assign heads of
Node* h2 = L2->gethead(); // respectively
```

```
int carry = 0;
```

```
while(h1 != N || h2 != N || carry != 0) // if all wrong - exit
{
    int sum = carry; // move prev carry to sum
```

```
    if(h1 != N)
```

```
    {
        sum += h1->data;
```

```
        h1 = h1->next;
```

```
    }
```

```
    if(h2 != N)
```

```
    {
        sum += h2->data;
```

```
        h2 = h2->next;
```

```
    }
```

```
    carry = sum / 10; // check if carry
```



```

int digit = sum % 10; //
result.insertion(digit); // mov dig to new list
}
return result; // between list when loop
// finishes
}

```

(DRY RUN)

Suppose this is our list

	L1	L2
0x1	1 0x2	0x4
0x2	2 0x3	0x5
0x3	3 N	0x6
		6 N

SSL result; // new obj

Node $\rightarrow$ h1 $\rightarrow$ 0x1 & h2 $\rightarrow$ 0x4
 while(h1 $\neq$ N || h2 $\neq$ N || carry $\neq$ 0) ✓
 {

int sum = carry; sum = 0;

if(h1 $\neq$ N) ✓
 {

sum += h1->d; sum = 1
 h1 = h1->next; h1 = 0x2

if(h2 $\neq$ N) ✓
 {

sum += h2->d; sum = 5
 h2 = h2->next; ~~sum~~ h2 = 0x5

carry = sum / 10; 5 / 10 = 0 // bec int dt

int dig = sum % 10; 5 % 10 = 5
 result.insertion(dig); 0x7 | 5 | N

again while(-) ✓
 {
 int sum = c;

s = 0

if(-) ✓
 {

s += hl->d;
 hl = hl->next;

s = 2

hl = 0x3

}
 if(-) ✓
 {

s += h2->d;
 h2 = h2->n;

s = 7

h2 = 0x8

}
 c = sum/10;

c = 0

int dig = sum%10

74010 = 7

result.insertion(dig);

0x9	5	0x8
0x8	7	N

again while(-) ✓
 {

int s = c;

s = 0

if(-) ✓
 {

s += hl->d;
 hl = hl->n;

3

N

if(-) ✓
 {

s += h2->d;
 h2 = h2->n;

9

N

c = s/10;

c = 0

int d = s%10;

d = 9

res.ins(d);

0x9	5	0x8
0x8	7	0x8
0x8	9	N

return result;

}

Question # 07

Node* findmid()

Node* slow = head;

Node* fast = head;

... explained before // Q # 03

Node* reverse(Node* node)

... explained before // Q # 01

~~Node*~~ void reverseList()

{
if (head == N || h->next == N) // 0 or 1 node
if yes → return
return;

Node* mid = findmid();
Node* sechalf = reverse(mid);

// store mid into
// reverse after mid

Node* fhalf = head;

// store head into fhalf

while (shalf->next != N) {

Node* t1 = fh->next;

Node* t2 = sh->next;

// closed in DRY RUN

fh->next = sh;

sh->next = t1;

fh = t1;

sh = t2; }

sh → next = t2;

fh = t1;
gh = t2;
fh = 0x3
gh = 0x3

1	0x5
5	0x5
2	0x4
4	0x3
3	N

while (→) X (sh → next = N)

(Question # 08)

void removeduplicates();

if (head == NULL) { return; }

Node* curr = head;

while (curr != NULL && curr → next != NULL) // exit if last or 2nd last

{ if (curr → data == curr → next → data) // if same data

{ Node* duplicate = curr → next; // move 2nd same to
curr → next = curr → next → next; dup & // print curr
delete duplicate; to dup's next
// del that node.

else

{ curr = curr → next; // if not same
iterate current.

}

DRY RUN

Suppose: our list is

1 1 2 2 3
0x1 0x2 0x3 0x4 0x5

Node = head: cur = 0x1

Now: while (cur != N && cur->next != N) ✓

{ if (cur->d == cur->next->data) ✓

{ Node* dup = cur->next; dup = 0x0
cur->next = cur->next->next; 0x1 -> 0x3
delete dup; }

else X

List Now

↓ 2 2 3
0x1 0x3 0x4 0x5

again: while (✓)

{ if (✓) 1 ≠ 2

else

{

c = c->n;

c = 0x3

}

again: while (✓)

{ if (✓) 2 = 2

dup = c->n;

dup = 0x4

c->n = c->n->n;

0x3 -> 0x5

{ del dup;

else (X)

List now 1 2 3

again: while (✓)

{ if (X) 2 ≠ 3

else

c = c->n;

c = 3

again

while (✓)

c->n =

Question # 09

void reverse()

checking edge case & calling helper function

void reverseHelper(Node* curr, int k)
 → head → node group to reverse

{
if (curr == NULL || k <= 1) // again checking edge cases for recursion
{
return curr;
}

Node* temp = curr;

int count = 0;

while (temp != NULL && count < k) // to check if we have k nodes to reverse
{

temp = temp->next;

count++;

}

if (count < k) { return reverseHelper(temp, k); }

Node* prev = NULL;

Node* current = curr;

Node* next = NULL;

count = 0;

while (—)

{ if (count > 0) }

if (next == NULL)

current->next = reverseHelper(next, k);

return prev; → new head

recursively calling if still nodes left

(DRY RUN)

Suppose: Our list is

$k = 2$

1 2 3 4

count = 0

// check if we have nodes to reverse k times (groups)
 $t = 0x1$, $curr = 0x1$
 $c < k$ ✓

1) while ($t \neq N$ & $c < k$) ✓
 {
 $t = t \rightarrow next$ $t = 0x2$
 $c++$ $c = 1$
 }

2) while (✓) ✓
 {
 $t = t \rightarrow n$ $t = 0x3$
 $c++$ $c = 2$
 }

3) while (✓) ✓
 {
 $t = t \rightarrow n$ $t = 0x4$
 $c++$ $c = 3$
 }

4) while (✓) ✓
 {
 $t = t \rightarrow n$ $t = 0x5$
 $c++$ $c = 4$
 }

5) while (✓) ✓
 {
 $t = t \rightarrow n$ N
 $c++$ $c = 5$
 }

6) while (→) ✗ $t = N$

Reverse DRY RUN → 1

→ 0x3 then 0x5 then exit func
 if (next != NULL) while (counting nodes)
 { reversing; } as it will be last node